May 2023

O2WB: A tool enabling ontology reuse in Wikibase

Daniil DOBRIY [a,1], Axel POLLERES [a]
[a] *WU Vienna University of Economics and Business*

Abstract.
Purpose: the Semantic Web initiative has established a set of standards and practices for publishing interconnected knowledge, where RDF Schema and OWL shall enable the reuse of ontologies as one of these established practices. However, notable Knowledge Graphs (e.g., Wikidata) do not currently support such schema reuse natively: in particular, Wikibase, the software behind Wikidata, which increasingly gains popularity among data publishers, lacks the functionality to import and reuse existing RDF Schema and OWL ontologies.
Methodology: in order to facilitate ontology reuse, we align the Wikibase data model with RDF, and present O2WB, a tool for ontology import & export within Wikibase. In a case study we demonstrate ontology reuse with O2WB.
Findings: in our case study we illustrate the effectiveness of our mapping between the only partially compatible RDF and Wikibase data models by importing an existing ontology into a Wikibase instance, editing it extensively and re-exporting it in a variety of formats.
Value: By enabling ontology reuse and editing within Wikibase, we facilitate a tighter connection of existing Linked Data resources with Wikibase instances.

Keywords. Wikibase, Linked Data, interoperability, ontology reuse

1. Introduction

The ability to reuse ontologies (schemas) is a defining feature and prerequisite of the Semantic Web [1]. Indeed, schema reuse is in line with Linked Open Data (LOD) principles [2], the reusability characteristic of the FAIR principles [3] as well as the 5th implementation level of the 5-Star Open Data deployment scheme.[2] Analyses of the openly published datasets that comply with Linked Data Principles in the manually curated Linked Open Data Cloud [4] have shown a significant number of links on the T-Box level (i.e. ontology links), as well as interlinks between the vocabularies themselves [5], indicative of widespread schema reuse, despite – however – significant availability issues [6] and a high number of ontology links being broken [7,8].

A recent analysis of such links specifically in Wikidata [9] has shown very few ontology links, which undermines the idea of Wikidata as a potential central data hub and entry point for LOD. Indeed, Wikidata's design steers clear of prior rigid ontological commitment that might impede the bottom-up development of Wikidata's scheme. This

[1] Corresponding Author
[2] http://www.w3.org/DesignIssues/LinkedData.html

design is buried deeply within the underlying data model and software, Wikibase, where (a) instance and schema data are not formally separated [10], and (b) functionality for schema reuse so far is largely lacking. This deliberate refusal of ontological commitment yet has its downsides: firstly, it forces Wikidata researchers to imply the schema by tracking proprietary taxonomic properties like *instance of* (P31) and *subclass of* (P279), instead of reusing respective RDF Schema and OWL [11,12,9]; secondly, the reuse of external namespaces and ontologies is practically not possible. All these factors, along with the differences between the RDF and Wikibase data models themselves, hinder direct schema reuse within Wikidata and other Wikibase instances.

In this context, we emphasize that also apart from Wikidata, platforms for collaborative maintaining of structured knowledge based on Wikis have started to be a significant driver for the creation of RDF datasets and Knowledge Graphs [13]: notable further examples for knowledge bases using Wikibase include the *EU Knowledge Graph* [14], the German National Library archive [15] and other instances showcased on the Wikibase website.[3] Another example is the still widely used Semantic MediaWiki (SMW) platform, in some sense the precursor to Wikibase; yet, as opposed to Wikibase, SMW supports ontology reuse through its more simple data model [16] as well as various tools available for imploring and exporting ontologies, e.g: *ontology2smw*[4] and *RDFIO*[5].

In this paper, we present a tool that enables such schema reuse also in Wikibase, with the final goal of potentially making Wikibase instances more discoverable and interconnected in the spirit of LOD.

As main contributions, (i) we recap/analyze the differences of the Wikibase and RDF data models, (ii) propose a mapping between these data models along with a schema to describe (RDFS and OWL) ontologies in Wikibase, and (iii) implement a tool, *O2WB*, that enables ontology reuse in Wikibase. Finally, (iv) we demonstrate round-trippable ontology import and export with our tool in a small, illustrating use case.

All resources, including the schema for representing ontologies in Wikibase and the O2WB tool, are publicly available via Anonymous GitHub[6] incl. documentation following the *Diátaxis* framework [7], as recommended by the Wikibase Stakeholders Group[8] for Open Source Wikibase-extensions. We plan to sustainably host and maintain the software through our institute, that already hosts various widely adopted Open Data and Semantic Web resources for several years now[9]; we further plan to engage in ongoing community activities such as the "Distributed Knowledge Graphs" COST Action[10], which as one of its goals aims at align, promote and sustain such community services and tools.

The remainder of this paper is organized as follows: first, in Section 2 we identify the differences between the RDF and Wikibase data models, and create a mapping for ontologies into Wikibase in Section 3. Then, in Section 4 we present O2WB. In Section 5, we demonstrate the ease of using it in a case study by importing an ontology from an existing website into Wikibase. In Section 6, we discuss the results and reference re-

[3] https://wikiba.se/showcase/
[4] https://github.com/andrecastro0o/ontology2smw
[5] https://github.com/rdfio/RDFIO/
[6] `https://anonymous.4open.science/r/o2wb`
[7] https://diataxis.fr
[8] https://wbstakeholder.group
[9] cf. `HTTP://URL_REMOVED_FOR_DOUBLE_BLIND_REVIEW`
[10] https://cost-dkg.eu/

```
@prefix : <http://alice.example.org#> .
T1 :me  rdf:type   foaf:Person .
T2 :me  foaf:name  "Alice Wonderland" .
T3 :me  foaf:knows [  foaf:name "Bob Builder" ] .
```

(a) Instance data

```
A1 foaf:Person rdfs:subclass foaf:Agent .
A2 foaf:name rdfs:range rdfs:Literal .
A3 foaf:knows rdfs:domain foaf:Person .
A4 foaf:knows rdfs:range foaf:Person .
```

(b) Ontology statements

Figure 1. RDF instance and ontology statements using the FOAF ontology.

lated works. Finally, in Section 7 we conclude the paper by identifying areas for further research.

2. Data model comparison

This section compares and maps the data model of Wikibase to the RDF data model. We start from an illustrating example in Figure 1 which shows a snippet consisting of FOAF (Friend of a Friend) profile, as well as part of the FOAF ontology, which we will employ in this and the following sections.[11]

The RDF data model [17] consists of ”flat” triples of the form

$$(I \cup B) \times I \times (I \cup B \cup L)$$

where I, B and L are disjoint sets of IRIs, Blank Nodes and Literals, respectively and Literals, where the set of Literals $L = Strings \times (D \cup Lang)$ could also be views as pairs of the actual lexical string and a resp. datatype (from D), again represented bu IRIs) or language tags ($Lang$). We left out datatypes and language tags, only using ”plain” Literals in the running example.

Wikibase's data model has a similar, but slightly different view centered around (also ternary) *Statements* of the form $(Q \cup P) \times P \times (Q \cup V)$ with

- Q and P not being arbitrary IRIs but disjoint sets of (sequentially) numbered Items $\{\texttt{Q1,Q2,...}\}$ and Properties $\{\texttt{P1,P2,...}\}$;
- V being Data-Values, similar to typed RDF Literals, but bound to a fixed, finite set of supported datatypes D_{wb}.
- the numbering being unique per Wikibase instance, i.e., e.g. the `P31` in Wikidata is not to meant to be the same Property in another Wikibase instance;
- labels L_Q and L_P of Items and Properties being first class citizens, i.e. each Item and Property are supposed to have a unique label, which, unlike RDF's commonly used `rdfs:label` property, is somewhat implicit.
- an emphasis on such Statements being *contextualisable*, e.g. ranking Statements about a single Statement as *preferred*, or providing contextual further meta-Statements (in separate namespaces) encoding e.g. provenance through qualifiers and separate reference Properties.

As for the latter point, Wikidata distinguished between so-called “truthy’ surface Statements and meta-Statements for such contextual information. Various works in the literature have already discussed different alternatives how to map this somewhat pre-

[11] for conciseness, we assume namespace non-defined prefixes according to `prefix.cc`.

defined meta-model of Wikibase/Wikidata to "flat" RDF triples [18], including extensions of RDF itself, such as RDF* [19] or novel unifying data models such as multilayer-graphs [20]. Herein, however, we are rather interested in the other direction, i.e. defining a bijective, i.e. round-trippable mapping from "flat" RDF to the Wikibase model.

In order to arrive at such a mapping, let us pinpoint the main obstacles and difference between these models:

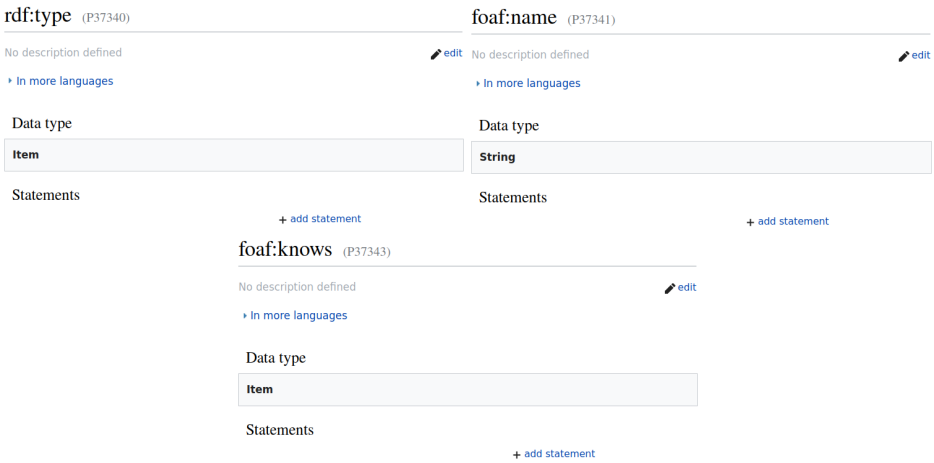
1. Wikibase only allows elements from $P \cup Q$ *defined within* a Wikibase instance to be subjects of (truthy) Statements, whereas RDF permits arbitrary IRIs as well as Blank Nodes (which stand as existential placeholders for things and concepts – Wikibase does not have Blank Nodes).
2. Wikibase allows only elements from P used in predicate position, whereas again RDF allows arbitraty IRI to be reused in any Triple position.
3. The UI and API of Wikibase imposes a de-facto strict typing of Properties on *data values* which, in contrast to RDF, limits each Properties to exclusivly point to Items, values from a pre-defined datatype (from a fixed set of allowed datatypes), or string Literals (with a specified language).

To illustrate these issues, let us start with a straightforward attempt to model the same data as in Fig. 1a, by simply trying to create a new Item `:me` along with triples `T1`–`T3` as Statements, in a plain Wikibase instance as illustrated in Fig. 2a+Fig. 2b.

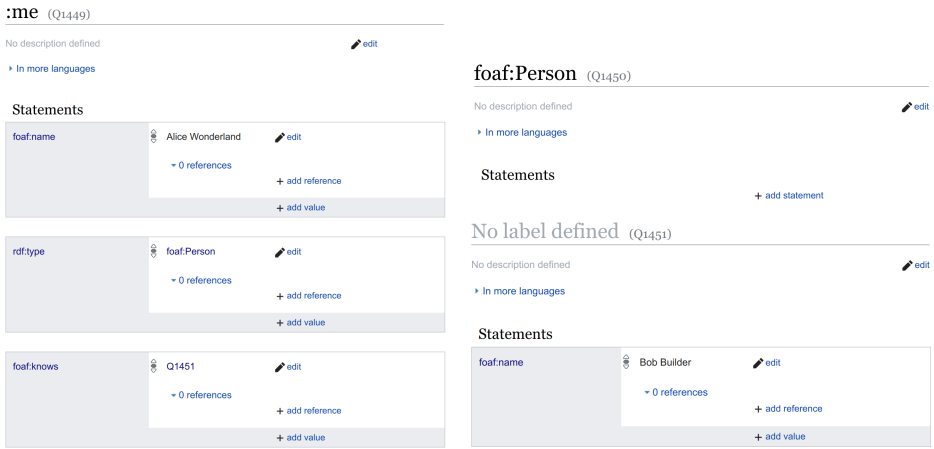
- the *Properties* need to be created first in order to be usable, and will receive sequential new numbers, e.g. `P37340`, `P37341`, and `P37343` resp., with corresponding declared datatypes, *String* and *Item*.

Fig. 2c shows the resulting default RDF export from Wikibase of this straightforward attempt: without even having imported or having thought of how to represent the ontology axioms, we already see some the mismatches here, e.g.

- having (prefix-abbreviated) IRI-references ending up as lang-tagged (with the default language for labels set in the Wikidata instance) string Literals, essentially loosing the references to external IRIs;
- extra meta-triples, using several Wikibase-specific namespaces for Properties, but also plain RDF (e.g. `rdfs:labels` and `rdf:type`), which should not interfere with our targeted import mapping.



(a) Modeling Properties in the Wikibase UI



(b) Modeling Items with Statements in the Wikibase UI

```
wd:Q1449  rdfs:label ":me"@en-gb ;  schema:dateModified "2023-05-23"^^xsd:dateTime .
wd:Q1449  wdt:P37341 "Alice Wonderland" ; wdt:P37340 wd:Q1450 ; wdt:P37343 wd:Q1451 .
...
wd:Q1450 rdfs:label "foaf:Person"@en-gb ; schema:dateModified "2023-05-23"^^xsd:dateTime .
...
wd:Q1451 schema:dateModified "2023-05-23"^^xsd:dateTime ; wdt:P37341 "Bob Builder" .
...
wd:P37340  rdfs:label "rdf:type"@en-gb ; schema:dateModified "2023-05-23"^^xsd:dateTime .
wd:P37340 rdf:type wikibase:Property .
```

(c) Wikibase RDF export for the above Items and Properties

Figure 2. Modeling the instance data from Fig. 1a in Wikibase

3. Ontology mapping

In order to define a bijective/round-trippable mapping for Wikibase (for both instance data and also ontology axioms) from and to RDF, we first establish a set of representation requirements that have to be met:

1. Imported classes (e.g. `foaf:Person`) and properties (e.g. `foaf:topic_interest`) should be usable inside Wikibase as Items and Properties;
2. The imported ontology should be editable in Wikibase with minimal restrictions;
3. An (edited) ontology from Wikibase should be exportable in the same form as it was imported.

To achieve these requirements, we propose to define a *meta-ontology* within Wikibase to identify Statements related to specific ontologies and distinguish them from "regular" Statements in order to be able to export ontologies or include new Items into them. Such a meta-ontology is defined in RDF and then represented in Wikibase according to the representation rules applicable to any ontology and discussed in the next section. The meta-ontology illustrated in Figure 3 is accessible (through content negotiation and with PyLODE documentation[12]) at and defines its classes and properties in the o2wb namespace URL[13].

Our ontology representation must sufficiently represent the features of the RDF data model not to lose any information necessary for later ontology export, or, respectively should mark the imported ontology statements, such that external references aren't easily accidentally modified altering their meaning within Wikibase. Still, in principle, our mapping relies on the idea of mapping the RDF representation of triples as well as (the RDF triple representation of) ontological axioms, to (truthy) Wikibase Statements.

To this end, firstly, all IRIs within the RDF terms of an imported RDF (instance data or ontology) graph are represented as both Items or Properties: since the RDF data model does not impose restrictions as Item/Property distinction and strict typing, the faithful representation of IRIs would allow them to be used in any capacity in accordance to the second requirement of allowing ontology management with minimal restrictions. Consequently, in order to allow the same "freedom" within Wikibase, any given IRI amongst the RDF terms of an imported RDF ontology or instance graph, has to be defined both as an Item and as (multiple) Properties (spanning every possible Wikibase Datatype). Although this representation of an IRI as multiple Properties spanning every Datatype in Wikibase would effectively already be sufficient for it to be used in any Statement position (subject, predicate, object as it is possible in the RDF data model) thus rendering representation of an IRI as an Item unnecessary, the over-use of Properties in subject position of Statements might lead to confusion as Items would normally be expected in this position and therefore we choose to also represent IRIs as Items.

For our example, that means the IRI `foaf:Person` will be represented by creating both an Item with a distinct Q-ID and multiple Properties with different P-IDs and datatypes (e.g. String, Monolingual Text, Item etc.[14]) within Wikibase.

[12] `https://github.com/rdflib/pyLODE`

[13] `HTTP://URL_REMOVED_FOR_DOUBLE_BLIND_REVIEW`
The anonymized version of the ontology is available at:
`https://anonymous.4open.science/r/o2wb/ontologies/o2wb.ttl`

[14] For all Wikibase datatypes see: https://www.wikidata.org/wiki/Help:Data_type

```
@prefix o2wb: <http://removed_double_blind/>.

o2wb:type a rdf:Property,
    owl:ObjectProperty;
    rdfs:comment "Rdfs:type equivalent.";
    rdfs:domain wikibase:Item;
    rdfs:isDefinedBy o2wb:;
    rdfs:label "type";
    rdfs:range rdfs:Class,
        owl:Class;
    vs:term_status "stable".

o2wb:Ontology a rdfs:Class,
    owl:Class;
    rdfs:comment "An ontology.";
    rdfs:isDefinedBy o2wb:;
    rdfs:label "Ontology";
    owl:equivalentClass owl:Ontology;
    vs:term_status "stable".

o2wb:BlankNodeRepresentation a rdfs:Class,
    owl:Class;
    rdfs:comment "Blank Node representation";
    rdfs:isDefinedBy o2wb:;
    rdfs:label "BlankNodeRepresentation";
    vs:term_status "stable".

o2wb:representsIRI a rdf:Property,
    owl:DatatypeProperty;
    rdfs:comment "Points to the IRI";
    rdfs:domain wikibase:Entity,
        o2wb:IRIRepresentation;
    rdfs:isDefinedBy o2wb:;
    rdfs:label "representsIRI";
    rdfs:range xsd:string;
    vs:term_status "stable".

o2wb:hasComplexAxiom a rdf:Property,
    owl:DatatypeProperty;
    rdfs:comment "Used to import Manchester syntax axioms.";
    rdfs:domain wikibase:Entity,
        o2wb:Ontology;
    rdfs:isDefinedBy o2wb:;
    rdfs:label "hasComplexAxiom";
    rdfs:range xsd:string;
    vs:term_status "stable".

o2wb:IRIRepresentation a rdfs:Class,
    owl:Class;
    rdfs:comment "IRI representation";
    rdfs:domain wikibase:Entity;
    rdfs:isDefinedBy o2wb:;
    rdfs:label "IRIRepresentation";
    rdfs:range rdfs:Class,
        owl:Class;
    vs:term_status "stable".

o2wb:fromOntology a rdf:Property,
    owl:ObjectProperty;
    rdfs:comment "Ontology embedding";
    rdfs:domain wikibase:Statement;
    rdfs:isDefinedBy o2wb:;
    rdfs:label "fromOntology";
    rdfs:range o2wb:Ontology;
    vs:term_status "stable".
```

Figure 3. O2WB meta-ontology

It is important to note here that the datatypes used in Wikibase for data values do not directly or approximately correspond to (e.g. XSD) datatypes commonly used in typed RDF Literals, since they represent more complex data structures (e.g. geographical shapes or coordinates).

We propose mapping RDF Literals occurring in triples to the Wikibase data model by representing their Turtle serialization [21] uniformly as the *String* datatype of Wikibase. Although this approach simplifies the variety of possible mappings of various datatype IRIs to Wikibase datatypes, it proposes a unified procedure that guarantees full freedom to modify Literal values represented in the Wikibase data model as they could be modified in the RDF data model. Further work may focus on establishing a more fine-grained mapping approach, e.g. suggesting an optional default mapping for compatible XSD and Wikidata datatypes.

Next, as *Blank Nodes* do not exist in the data model of Wikibase, they therefore have to be represented as ”Skolem-Items”, simple being assigned a new dedicated Q-ID, with otherwise the same representation in Wikibase as previously discussed for IRIs.

Finally, since the original IRIs and "skolemized" Blank Nodes are replaced by different Item and Property Q-IDs and P-IDs within Wikibase, in order to maintain ontological interoperability, equivalence to the original IRIs, or reference as originating from a Blank Node has to be established: to this end, upon import all newly added Items and Properties, depending on whether they stem from imported RDF IRI or Blank Nodes, get an extra Wikibase Statement using either the dedicated meta-Property `o2wb:representsIRI` with a *String* data value of the original IRI, the meta-Property `o2wb:type` with the value `o2wb:BlankNodeRepresentation`.

Additional meta-ontological concepts to be defined are the `o2wb:Ontology` class which shall represent an imported ontology and `o2wb:fromOntology` represented as Property with datatype Item used in a qualifier to indicate that a given Statement is a representation of a Triple from a given ontology. For each ontology import, an ontology individual of class o2wb:Ontology has to be established and represented in Wikibase for all the related Statements to be set in `o2wb:fromOntology` relation with it.

We illustrate the import result of such an import in Fig. 4, by showing how the imported IRIs, Blank Nodes and Statements corresponding to the RDF triples in Fig. 1a appear in Wikibase using O2WB – as opposed to the naive representation from Fig. 4.

As we can see in Fig. 4, using these additions, all original triples can be losslessly represented as Wikibase Statements. IRIs or Blank Nodes in subject position of a Triple are represented as the corresponding equivalent Wikibase (o2wb:representsIRI) Items in subject position of a Statement. IRIs in predicate position of a Triple are represented as the respective Properties with either datatype Item or String in predicate position of a Statement depending on whether the object of a Triple is either an IRI/Blank Node, or a Literal. IRIs or Blank Nodes in object position of a Triple are represented as Items and Literals are represented as DataValues with datatype String in the object position of a Statement.

Finally, the (axiom) triples such as `A1` from Fig. 1b:

```
foaf:Person rdfs:subClassOf foaf:Agent .
```

would be likewise translated into Wikibase Statements analogously: for `A1`, with the Item representing `foaf:Person` in its subject position, the Property with datatype *Item* representing `rdfs:subClassOf` in predicate position and the Item representing `foaf:Agent` in object position. Similarly, the triple

```
foaf:myersBriggs vs:term_status "testing" .
```

also appearing in the FOAF ontology, would be imported into a Statement that has the (Q-)Item representing `foaf:myersBriggs`, the (P-)Property with datatype *String* representing `vs:term_status` and a string `"testing"` for the Literal.

In this way, even complex (e.g. ternary) OWL axioms would be represented in their RDF notation. However, as illustrated in 5a, such verbose representation makes editing complex axioms cumbersome in the Wikibase GUI. We address this in a twofold manner: not recommending the editing of very complex OWL axioms in Wikibase on one hand and proposing an alternative approach specifically tailored to this case and available as an option in O2WB.

Following this approach, complex multi-triple axiomatic structures (e.g. OWL restrictions) are represented separately as String values in their Manchester notation [22] as

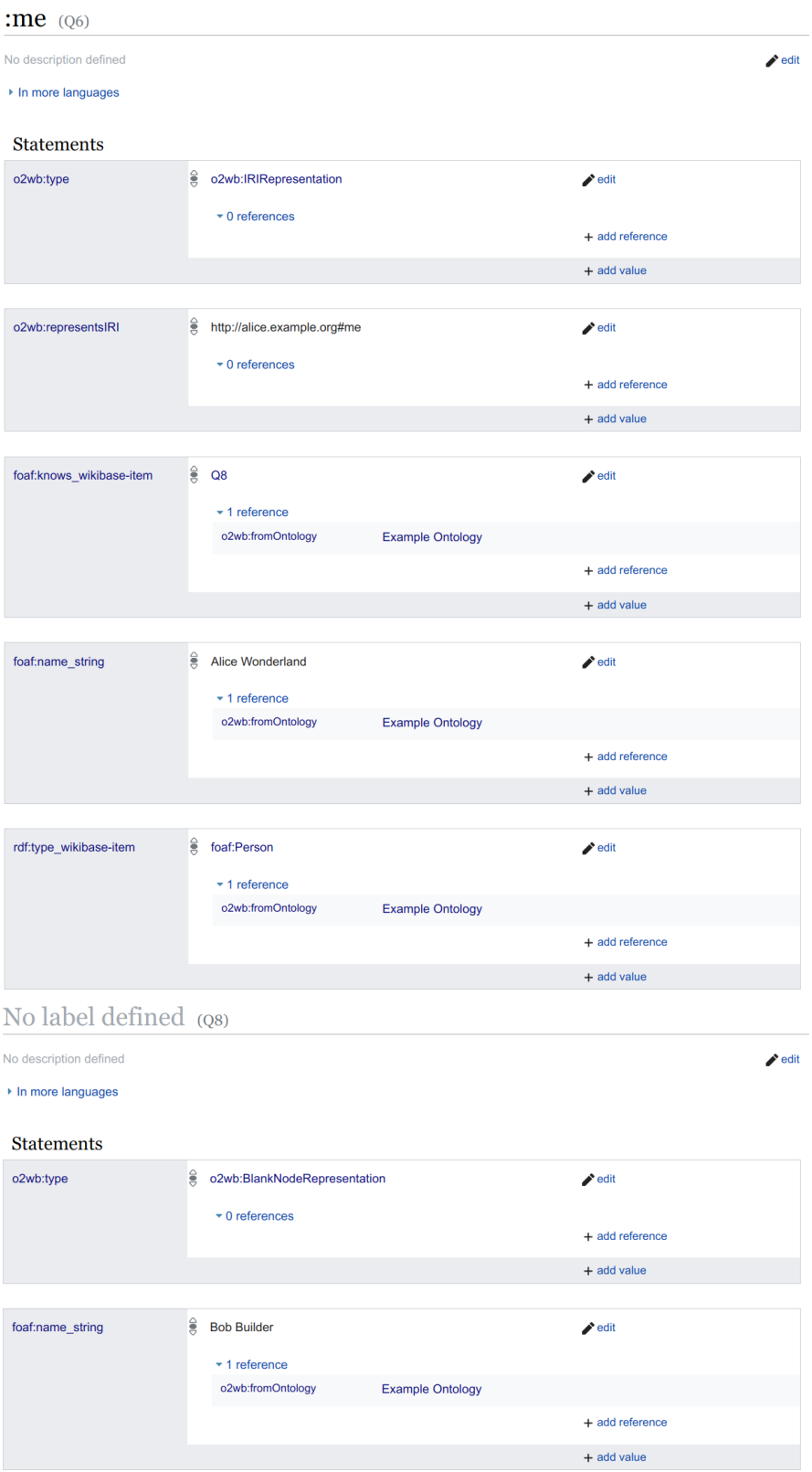


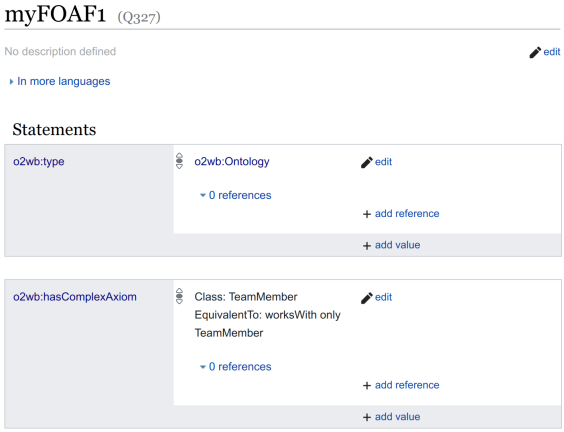
Figure 4. Import result using O2WB: compare the Items with extra Statements in the Wikibase UI with the naive representation from Fig 2c

```
:TeamMember rdf:type owl:Class ;
    owl:equivalentClass [
        rdf:type owl:Restriction ;
        owl:onProperty :worksWith ;
        owl:allValuesFrom :TeamMember
    ] .
```

(a) Turtle syntax

```
Class: TeamMember
    EquivalentTo: worksWith only TeamMember
```

(b) Manchester syntax



(c) Manchester syntax in Wikibase

Figure 5. Representing complex axioms in Wikibase

seen in Figure 5b and related to the respectful ontology via the `o2wb:hasComplexAxiom` Property as illustrated in Figure 5c. In this way, they can be directly edited in the sparser Manchester notation.

4. Description of the O2WB tool

The O2WB ontology import tool,[15] licensed under the CC0 1.0 Universal license,[16] is a Python command line utility that allows you to import ontologies into Wikibase from a local file or a URL. It is based on the *rdflib* and *wikibase-integrator* libraries. The former provides an internal structure for the handling of RDF graphs and the latter simplifies interaction with the Wikibase API.

The tool follows the approach described in the previous sections: For each ontology to be imported, O2WB creates an Item in Wikibase representing the ontology and iterates over each triple/axoim of an ontology creating appropriate representations of encountered IRIs and Blank Nodes if they do not exist yet: simple (single-triple) TBox axioms or ABox facts are instantiating Statements that correspond to the triples using the newly created representations with correct datatypes and including qualifiers to indicate that the Statements belong to the imported ontology, whereas complex ontology axioms are imported as strings, as described above. Ontologies imported with help of this approach

[15] `https://anonymous.4open.science/r/o2wb`
[16] `https://creativecommons.org/publicdomain/zero/1.0/`

can be effectively edited though the Wikibase GUI and exported back. The edit history is then preserved as page edit logs for the respective entities.

Some of the notable features of the tool include the ability to also recursively import ontologies defined via `owl:imports` statements (`--recursive`), import and export ontologies in a variety of formats which include all common RDF serializations as well as a mapping schema generator that creates a comma-separated reference with original IRIs ad their respective IDs (`--mapping`). These and other features are described and illustrated in more detail in the next section.

5. Case study: O2WB in action

In this section we demonstrate how to import an ontology into Wikibase, extend and edit it using Wikibase GUI and then export the edited ontology back to various serializations. We will walk though every step of the process and by the end of this study you should have a good understanding of ontology management with Wikibase.

5.1. Setting up a Wikibase Suite instance

Before we can start importing an ontology into Wikibase, we need to set up a Wikibase instance, also known as the Wikibase Suite[17]. The architecture of the Wikibase Suite consists of the MediaWiki base platform with Wikibase extensions, the MySQL (MariaDB) database and the Elasticsearch, QuickStatements and Wikidata query services[18]. There are two main approaches to setting up Wikibase Suite: a Docker-based setup and a full manual setup.

In the Docker-based setup, Docker containers with the mentioned components are set up automatically, simplifying most of the installation and configuration tasks. In contrast, the full manual setup requires separate installation of each component. In the context of this case study we recommend users to follow the Docker-based setup described in this manual[19] unless you have specific local requirements that are otherwise not met. Make sure to replace the data in the `.env` configuration file with your own and inspect that the configured ports for services are indeed available.

Once you have the Wikibase suite up and running, you can access the Wikibase UI by visiting the URL of your MediaWiki installation in a web browser. After logging in using details from the configuration file, you can use various *Special Pages* to administer your Wikibase instance. Navigate to the *Bot Passwords* Special Page and create a bot with the additional permissions of: *High-volume editing*, *Edit existing pages* and *Create, edit, and move pages*. If you encounter any issues while setting up your Wikibase instance, you can refer to the troubleshooting section of the manual, seek help from the Wikibase user community[20].

[17] `https://www.mediawiki.org/wiki/Wikibase/Suite`
[18] `https://www.mediawiki.org/wiki/Wikibase/Maintaining#Architecture_overview`
[19] `https://www.mediawiki.org/wiki/Wikibase/Docker`
[20] `https://lists.wikimedia.org/postorius/lists/wikibaseug.lists.wikimedia.org/`

5.2. Importing an ontology with O2WB

Now that we have the Wikibase instance set up and the bot configured, we can start importing an ontology with O2WB. To use the O2WB ontology import tool, you can install it by cloning the O2WB repository[21] from Anonymous GitHub and following the README instructions to install the dependencies and modify the configuration file with bot details generated in Subsection 5.1.

Once O2WB is set up and configured, open a terminal window and use the following command to import an ontology:

```
python import.py {--url <URL of your ontology> | --file <path to the
ontology file>} --name <ontology name> [--recursive <Depth of
recursive imports>] [--mapping <path to export the mapping scheme>
--complexAxioms]
```

With O2WB, you can either import an ontology from a URL or from a local file. The tool supports all common RDF serializations[22] and can recursively import ontologies mentioned in `owl:imports` statements. When you run the command, O2WB will begin processing the ontology and adding it to your Wikibase instance. In this case study, we use the following command to import the example ontology:

```
python import.py --file ontologies/myFOAF.ttl --name "myFOAF1"
```

The import process may take a few minutes to complete depending on the size of the imported ontology and available processing resources. Progress bar shows the current status of ontology import in the terminal window. Once the process is complete, you will see a summary message with the Q-ID of the Item representing the imported ontology as well as the number of Items, Properties and Statements that were created:

```
O2WB succesfully imported myFOAF1 (Q327)!
New Items: 10
New Properties: 20
New Statements: 100
```

You can now navigate to the *Recent Changes* Special Page to browse Entities that O2WB added to your Wikibase instance. If you encounter any issues or errors while importing an ontology, you can refer to the O2WB documentation, create an issue on the O2WB GitHub repository or contact the authors directly for support.

5.3. Editing an ontology with Wikibase UI

After having imported the ontology, we now can edit it using the Wikibase UI. There are three types of edits that we look at: adding new Statements, editing existing Statements and deleting Statements.

[21] `https://anonymous.4open.science/r/o2wb`

[22] For supported serializations see: `https://rdflib.readthedocs.io/en/stable/plugin_serializers.html`

To add a new Statement, first think about all the Entities that make it up and their datatypes in Wikibase. If an Entity does not exist yet or there is no Property that is typed correctly, you should create it through either the *Create a new Item* or *Create a new Property* Special Page. When creating a new Entity, use the `o2wb:representsIRI` Property to indicate the IRI that will be exported later or the `o2wb:type` Property with the value of `o2wb:BNodeRepresentation` to indicate that the Entity is a Blank Node.

After making sure that all the necessary Entities exist, navigate to the page of the Entity that is supposed to be the subject of the new Statement. On this page you can add a new Statement selecting the already existing Entities. To include the recently created Statement in an ontology, add a Reference with the Property `o2wb:fromOntology` and an Item that stands for the respective ontology.

In a similar manner, you can edit Statements by editing or replacing their constituent Entities or chaining their non-Entity Values. When replacing an Entity, make sure that it is created beforehand and has an appropriate datatype. Finally, you can exclude Statements from a given ontology by deleting them or removing their `o2wb:fromOntology` references.

5.4. Exporting an ontology with O2WB

Exporting an ontology presents the last step of the ontology management process. Open the terminal window and use the O2WB *export* command to export an ontology from your Wikibase instance. As you can see below, it largely follows the same syntax as the *import* command:

```
python export.py --file <path to ontology file> --name <ontology
name> [--mapping <path to export the mapping scheme>
--complexAxioms]
```

A bar will appear tracking the progress of the export process. After a successful export, the ontology can be further shared and reused in other environments, facilitating the reuse of ontologies on the Semantic Web.

In our example, we can compare the edited version of the ontology in Figure 6 with the initial one as depicted in Figure 1. Note that the categorization in the instance (ABox) and ontology (TBox) parts is done by the authors and does not affect the import and export process. For illustration, the ontology has been exported in Turtle serialization, although the tool also supports exports in any other common serialization.

Insignificant differences to the initial ontology file such as the changes prefix labels or variations in the use of Turtle abbreviations are not considered material to the content itself (O2WB simply generates a consecutive prefix abbreviations for each namespace prefix appearing multiple times), apart from that the extracted set of RDF triples is exactly equivalent to the original imported file.

6. Discussion and related works

Although some work has described importing various data into a Wikibase instance [23, 24,25], to the best of our knowledge no known work has specifically targeted ontology reuse in Wikibase under considerations of data the model disparity with RDF in order to

```
@prefix ns1: <http://xmlns.com/foaf/0.1/> .
@prefix rdfs: <http://www.w3.org/2000/01/
rdf-schema#> .

<http://alice.example.org#me> a ns1:Person ;
    ns1:knows [ ns1:name "Bob Builder" ] ;
    ns1:name "Alice Wonderland" .
```

(a) Instance data

```
ns1:knows rdfs:domain ns1:Person ;
    rdfs:range ns1:Person .

ns1:name rdfs:range rdfs:Literal .

ns1:Person rdfs:subclass ns1:Agent .
```

(b) Ontology Statements

Figure 6. Exported RDF instance and ontology Statements

systematically increase ontology interoperability in the Wikibase ecosystem. On the contrary, similar in spirit tools have been developed for Semantic MediaWiki [26], which in some sense may be viewed as Wikibase's predecessor. However, since the design choices are strongly dependent on the underlying data model of the system, SMW solutions are not directly transferable to Wikibase, and particularly, we are not aware of other works having proposed a solution for data model reconciliation between Wikibase and RDF for the purpose of allowing ontology management. Rather, the topic of mapping the data model of Wikibase has merely been discussed in the other direction, proposing alternative RDF mappings for exporting (both truthy and particularly meta-)Statements from Wikibase [18], using different reification approaches.

We believe that the other direction, namely enabling the ability to automatically import and then manage ontologies within Wikibase further qualifies Wikibase as a viable ontology management software [27]. By facilitating the reuse of ontologies, such an approach also contributes to aligning Wikibase with last points of both the Linked Data Principles (LD4) and the 5-Star Deployment Scheme (5ST). Through the use of ontological terms with IRIs from a external namespace in non-instance position, users can create external ontology links [8] and increase the degree of Linked Data inter-linkage from Wikibase instances such as Wikidata in terms of enabling ontology links, an issue that we have have earlier criticised [9].

7. Conclusions and Future Work

We have presented a tool, O2WB, which shows that it is possible to faithfully import and maintain an ontology in Wikibase in a way that will also allow its export at a later stage. The Wikibase data model possesses a number of unique features that potentially limit its ontological interoperability. To still ensure a faithful representation of ontologies in the Wikibase data model, we represent IRIs from imported "plain" RDF graphs (including RDFS and OWL ontologies) as both Wikidata *Items* as well as multiple *Properties* spanning all allowed Wikidata datatypes; Blank Nodes in the original graph are skolemized before being represented analogously to IRIs. We presented a supporting ontology to be used to mark/identify ontology contents from imported RDF graphs, in the form of additional meta-Statements, which do not limit further editing of the imported ontologies within Wikibase. Finally, we presented the usage of our O2WB tool as an implementation of the discussed approach to mapping and reusing ontologies within Wikibase.

Lastly, as an extension for future work, we note that our mapping of datatype Literals is so far somewhat limited: although Wikibase datatypes comprise more complex

data structures than the XSD datatypes, their equivalents could be found in structures defined by other specialized domain ontologies[23]. Future work could, besides mappings for compatible XSD datatypes, include mappings between Wikibase datatypes to equivalent structures in commonly used ontologies for the respective domains. Further, while currently O2WB ensures the faithful representation of the RDF data model through a palette of corresponding entities, a future implementation could additionally offer a more differentiated representation that would allow to assign specific datatypes to imported Properties. Finally, instead of our proprietary commandline export tool, likewise bespoke SPARQL queries could be defined to directly export a given ontology, or, respectively views on specific imported ontologies, in their original "flat" RDF representation, via the Wikibase instance's SPARQL endpoint – which would also allow non-administrators to re-export imported ontologies from a running Wikibase instance that deploys our extension.

References

[1] Simperl E. Reusing ontologies on the Semantic Web: A feasibility study. Data & Knowledge Engineering. 2009;68(10):905-25.

[2] Bizer C, Heath T, Berners-Lee T. Linked data: The story so far. In: Semantic services, interoperability and web applications: emerging concepts. IGI global; 2011. p. 205-27.

[3] Wilkinson MD, Dumontier M, Aalbersberg IJ, Appleton G, Axton M, Baak A, et al. The FAIR Guiding Principles for scientific data management and stewardship. Scientific data. 2016;3(1):1-9.

[4] et al JM. The Linked Open Data Cloud; 2007. Https://lod-cloud.net/ (Last accessed 23 May 2023).

[5] Vandenbussche PY, Atemezing G, Poveda-Villalón M, Vatant B. Linked Open Vocabularies (LOV): A gateway to reusable semantic vocabularies on the Web. Semantic Web Journal. 2017;8(3):437-52.

[6] Polleres A, Kamdar MR, Fernández JD, Tudorache T, Musen MA. A More Decentralized Vision for Linked Data. In: Verborgh R, Kuhn T, Berners-Lee T, editors. Proceedings of the 2nd Workshop on Decentralizing the Semantic Web co-located with the 17th International Semantic Web Conference, DeSemWeb@ISWC 2018, Monterey, California, USA, October 8, 2018. vol. 2165 of CEUR Workshop Proceedings. CEUR-WS.org; 2018. .

[7] Neumaier S, Polleres A, Steyskal S, Umbrich J. Data integration for open data on the web. In: Reasoning Web International Summer School. Springer; 2017. p. 1-28.

[8] Haller A, Fernández JD, Kamdar MR, Polleres A. What are Links in Linked Open Data? A Characterization and Evaluation of Links between Knowledge Graphs on the Web. ACM Journal of Data and Information Quality (JDIQ). 2020 May;2(2):1—34.

[9] Haller A, Polleres A, Dobriy D, Ferranti N, Rodríguez Méndez SJ. An Analysis of Links in Wikidata. In: European Semantic Web Conference. Springer; 2022. p. 21-38.

[10] Piscopo A, Simperl E. Who models the world? Collaborative ontology creation and user roles in Wikidata. Proceedings of the ACM on Human-Computer Interaction. 2018;2(CSCW):1-18.

[11] Erxleben F, Günther M, Krötzsch M, Mendez J, Vrandečić D. Introducing Wikidata to the linked data web. In: International semantic web conference. Springer; 2014. p. 50-65.

[12] Brasileiro F, Almeida JPA, Carvalho VA, Guizzardi G. Applying a multi-level modeling theory to assess taxonomic hierarchies in Wikidata. In: Proceedings of the 25th International Conference Companion on World Wide Web; 2016. p. 975-80.

[13] Tramullas J, Sánchez-Casabón AI, Garrido-Picazo P. Organización y descripción de información en wikis semánticos: Wikibase, Semantic Mediawiki y Cargo. 2021.

[14] Diefenbach D, Wilde MD, Alipio S. Wikibase as an Infrastructure for Knowledge Graphs: The EU Knowledge Graph. In: International Semantic Web Conference. Springer; 2021. p. 631-47.

[15] Fischer B. Towards an open and collaborative Authority Control. Towards an open and collaborative Authority Control. 2022:283-90.

[23] E.g., the Wikibase datatype *Geographic shape* could be expressed by *GeoXG* or other geospatial ontologies, etc.

[16] Vrandečić D, Krötzsch M. Wikidata: a free collaborative knowledgebase. Communications of the ACM. 2014;57(10):78-85.

[17] Manola F, Miller E, McBride B, et al. RDF primer. W3C recommendation. 2004;10(1-107):6. Https://www.w3.org/TR/rdf-primer/ (Last accessed 23 May 2023).

[18] Hernández D, Hogan A, Krötzsch M. Reifying RDF: What works well with wikidata? SSWS@ ISWC. 2015;1457:32-47.

[19] Hartig O. Foundations of RDF⋆ and SPARQL⋆ (An Alternative Approach to Statement-Level Metadata in RDF). In: Proceedings of the 11th Alberto Mendelzon International Workshop on Foundations of Data Management and the Web. vol. 1912 of CEUR Workshop Proceedings. CEUR-WS.org; 2017. Available from: `http://ceur-ws.org/Vol-1912/paper12.pdf`.

[20] Angles R, Hogan A, Lassila O, Rojas C, Schwabe D, Szekely PA, et al. Multilayer graphs: a unified data model for graph databases. In: GRADES-NDA@ SIGMOD; 2022. p. 11-1.

[21] Prud'hommeaux E, Carothers G. RDF 1.1 Turtle-Terse RDF Triple Language, W3C Recommendation 25 February 2014. W3C Recommendation. World Wide Web Consortium (W3C) http://www w3 org/TR/2014/REC-turtle-20140225. 2014.

[22] Horridge M, Patel-Schneider PF. OWL 2 web ontology language manchester syntax. W3C Working Group Note. 2009.

[23] Pellissier Tanon T, Vrandečić D, Schaffert S, Steiner T, Pintscher L. From freebase to wikidata: The great migration. In: Proceedings of the 25th international conference on world wide web; 2016. p. 1419-28.

[24] Shigapov R, Mechnich J, Schumm I. RaiseWikibase: Fast inserts into the BERD instance. In: European Semantic Web Conference. Springer; 2021. p. 60-4.

[25] Shigapov R, Zumstein P, Kamlah J, Oberländer L, Mechnich J, Schumm I. bbw: Matching CSV to Wikidata via Meta-lookup. In: CEUR Workshop Proceedings. vol. 2775. RWTH; 2020. p. 17-26.

[26] Castro A, Strömert P. Importing RDF ontologies to SMW with ontology2smw and the Academic Event Ontology; 2020. Https://www.semantic-mediawiki.org/w/images/2/24/Ontology2smw_smwcon.pdf (Last accessed 23 May 2023).

[27] Bloehdorn S, Haase P, Huang Z, Sure Y, Völker J, Harmelen Fv, et al. Ontology management. In: Semantic knowledge management. Springer; 2009. p. 3-20.